

Master Question Bank

Computer Organization & Architecture

Contents

1	Number Representation and Boolean Algebra	3
1.1	Bit-Width and Precision Analysis	3
1.2	Number Representation Theory	3
1.3	Number Systems and IEEE-754 (Long-form)	3
1.4	Boolean Algebra Proofs (Long-form)	4
2	Combinational Logic Design	4
2.1	MUX, Encoder, Decoder, and Display Design	4
2.2	Half Adders and Full Adders (Long-form)	4
2.3	Latches, Multiplexers, and Encoders (MCQ)	5
2.4	Adder Analysis (MCQ)	7
2.5	Full Adders and Carry-Lookahead Adders (MCQ)	7
3	Sequential Logic	8
3.1	Flip-Flops (MCQ)	8
4	Memory Systems	10
4.1	Memory and Address Bus	10
4.2	SRAM (MCQ)	10
4.3	DRAM (MCQ)	12
4.4	Memory Cell Comparison (MCQ)	13
4.5	Memory Hierarchy and Locality (MCQ)	13
4.6	Cache Organization (MCQ)	15
4.7	Memory Cells (Long-form)	16
5	Instruction Set Architecture and Microprogrammed Control	16
5.1	Instruction Formats	16
5.2	PC-Relative Addressing (Long-form)	17
5.3	Microprogrammed Control Units	18
6	SimpleRisc Processor	19
6.1	SimpleRisc Pipeline (MCQ)	19
6.2	SimpleRisc Instruction Encoding and Control (MCQ)	21
7	Pipelining	24
7.1	Pipelining Fundamentals (MCQ)	24

7.2	Pipelined Processor Datapath (MCQ)	25
7.3	Pipeline Hazards — Detailed Scenarios (MCQ)	27
7.4	Pipeline Mathematical Model (MCQ)	30
7.5	Pipeline Execution Problems (Numerical)	31
8	Assembly Programming	33
9	Theory of Computation	33
9.1	Turing Machines	33

1 Number Representation and Boolean Algebra

1.1 Bit-Width and Precision Analysis

1. **P1.1** Let A and B be two n -bit unsigned integers. Derive the maximum possible value of $A + B$ and determine how many bits are required to represent the sum without overflow.
2. **P1.2** Suppose a system performs repeated additions of K unsigned n -bit integers. Derive the minimum bit-width required to store the exact sum.
3. **P2.1** Suppose each product term is guaranteed to be at most 2^p . If K such terms are summed, derive the minimum number of bits needed to store the exact sum.
4. **P2.2** Assume a fixed accumulator of width m bits is used. Derive the maximum value of K such that summing K numbers each of value 2^p does not overflow.
5. **P2.3** Conceptually explain why increasing numerical precision in AI systems is fundamentally a hardware resource tradeoff, using only bit-level reasoning.

1.2 Number Representation Theory

1. Consider the `sbm` instruction that subtracts the second operand from the first operand, and branches to the instruction specified by the label (third operand) if the result is negative. Write a small program using only the `sbm` instruction to compute the factorial of a positive number.
2. Show that any n -bit unsigned integer can be incremented using only bit flips and a carry signal.
3. Let S_n be the set of all n -bit binary strings. What is $|S_n|$?
4. How many distinct Boolean functions $f : \{0, 1\}^n \rightarrow \{0, 1\}$ exist?
5. Suppose a computation requires representing integers up to M . Derive the minimum number of bits required.
6. Derive the condition under which unsigned addition overflows.

1.3 Number Systems and IEEE-754 (Long-form)

1. Consider the equality $(43)_x = (y3)_8$ where x and y are unknown. What are the possible solutions of (x, y) ? (2 marks)
2. Consider the IEEE-754 single precision floating point numbers $P = 0xC1800000$ and $Q = 0x3F5C2EF4$. What is the product of these numbers (i.e., $P \times Q$), represented in the IEEE-754 single precision format? (3 marks)
3. Perform the following subtraction operations using the r 's complement method. For each case, explicitly show the calculation of the complement for the subtrahend and the subsequent addition step. (3 + 4 = 7 marks)
 - (i) $(101.101)_2 - (11.011)_2$ [Use 2's complement; express your final answer in base-2]
 - (ii) $(256.34)_7 - (143.56)_7$ [Use 7's complement; express your final answer in base-7]

4. What is the base of the number system in which the following equation holds?

$$\frac{312}{20} = 13.1$$

(2 marks)

1.4 Boolean Algebra Proofs (Long-form)

1. Prove each of the following Boolean expressions using Boolean algebraic laws. (2 × 2 = 4 marks)

(i) $\overline{(A + \bar{B} + \bar{D})(C + D)(\bar{A} + C + D)(A + B + \bar{D})} = \bar{A}D + \bar{C}\bar{D}$

(ii) $(A + C)(\bar{A} + B) = AB + \bar{A}C$

2. Prove each of the following Boolean expressions using Boolean algebraic laws. (2.5 × 2 = 5 marks)

(i) $(A' + B') \cdot (A' + B) \cdot (A + B' \cdot D + C) = A' \cdot B' \cdot D + A' \cdot C$

(ii) $A \cdot B' + A' \cdot B' + A \cdot B' \cdot C \cdot D = B'$

2 Combinational Logic Design

2.1 MUX, Encoder, Decoder, and Display Design

- Design a 4-to-1 multiplexer using only 2-to-1 multiplexers.
- Implement an 8-to-1 multiplexer utilizing a combination of 4-to-1 and 2-to-1 multiplexers.
- Design a combinational logic circuit to drive a 7-segment display for decimal representation.
- Design a parity bit checker and a digital magnitude comparator.

2.2 Half Adders and Full Adders (Long-form)

- If a half adder is designed using only NAND gates, and a single NAND gate delay is $2\mu s$, what is the minimum possible propagation delay for the half adder? What is the minimum number of NAND gates required to realize it?
- Can you design a full adder using half adders (H.A.)?
- A half adder is implemented using an XOR gate and an AND gate. A full adder is implemented using 2 half adders and one OR gate. Given:

$$P_{\text{XOR}} = 2 P_{\text{AND}}, \quad P_{\text{AND}} = P_{\text{OR}} = 1.2\mu s,$$

determine the propagation delays of the half adder and full adder.

- Draw the circuit diagram of a standard 4-bit ripple-carry adder. Then, explain how you can configure or augment this exact same adder circuit (without fundamentally changing its internal structure) to perform the following specific operations on a 4-bit input A :
 - 2's Complement Subtraction:** Computing $A - B$ (where B is a second 4-bit input).

- (ii) **Increment:** Computing $A + 1$.
- (iii) **Decrement:** Computing $A - 1$.

2.3 Latches, Multiplexers, and Encoders (MCQ)

1. In a standard NAND SR latch configuration, what happens to the stored data when the inputs are dropped to $S = 0$ and $R = 0$?
 - (a) The latch resets
 - (b) The latch sets
 - (c) The latch maintains its old values
 - (d) The latch enters a forbidden state
2. If a demultiplexer has 5 select bits, what is the maximum number of output lines it can support?
 - (a) 5
 - (b) 10
 - (c) 25
 - (d) 32
3. What is the minimum number of 2:1 MUXs required to design a 4:1 MUX?
4. In the logical construction of an 8-to-3 encoder, multiple input lines are fed into OR gates to generate the binary output. Which specific input bits determine the least significant bit (Bit0) of the output?
 - (a) b_4, b_5, b_6, b_7
 - (b) b_2, b_3, b_6, b_7
 - (c) b_1, b_3, b_5, b_7
 - (d) b_0, b_2, b_4, b_6
5. An 8-to-3 encoder's output is directly wired to the input of a 3-to-8 decoder. If the 4th input line (b_4) of the encoder is activated, which output line of the decoder will subsequently be set to 1?
 - (a) Line 2
 - (b) Line 3
 - (c) Line 4
 - (d) Line 8
6. What is the propagation delay for a binary CLA (carry-lookahead) adder where the inputs are 4-bit numbers?

7. A processor needs to read data from one of 250 distinct memory banks. It uses a multiplexer to route the data from the chosen memory bank to the CPU. What is the minimum number of select lines this multiplexer must have to handle all 250 inputs?
- (a) 7
 - (b) 8
 - (c) 9
 - (d) 250
8. A clocked NAND SR latch requires a clock signal to update its state. If the clock signal is currently 0, how does the latch respond to changes on the S and R input lines?
- (a) It behaves exactly like a classic SR latch.
 - (b) It forces the outputs to 0.
 - (c) It enters a high-impedance state.
 - (d) It ignores the inputs and maintains its old values.
9. An encoder takes n inputs (assuming only one is active) and finds its binary ID. If the 5th input out of 8 inputs is the only one set to 1, what will the 3-bit binary output (Bit2, Bit1, Bit0) be?
- (a) 100
 - (b) 101
 - (c) 110
 - (d) 111
10. When evaluating the logical states of a NAND SR latch, which input combination is strictly avoided because it leads to an undefined state where the outputs do not properly complement each other?
- (a) $S = 0, R = 0$
 - (b) $S = 0, R = 1$
 - (c) $S = 1, R = 0$
 - (d) $S = 1, R = 1$
11. In a clocked NAND SR latch, the clock transitions to 1. While the clock is 1, the inputs are set to $S = 0$ and $R = 1$. What is the resulting state of Q ?
- (a) $Q = 0$
 - (b) $Q = 1$
 - (c) Q toggles based on the clock frequency.
 - (d) Q maintains its previous state.

2.4 Adder Analysis (MCQ)

1. What is the asymptotic time complexity of an n -bit Ripple Carry Adder?
 - (a) $O(\log n)$
 - (b) $O(1)$
 - (c) $O(n^2)$
 - (d) $O(n)$
2. For an n -bit Carry Select Adder divided into blocks of equal size k , what is the mathematically optimal block size k to minimize the total computation time?
 - (a) $k = n/2$
 - (b) $k = \log(n)$
 - (c) $k = n^2$
 - (d) $k = \sqrt{n}$
3. If an n -bit Carry Select Adder is divided into individual blocks of size k , how much computational time does Stage II (the sequential selection stage) take to complete?
 - (a) $n \cdot k$ units of time
 - (b) n/k units of time
 - (c) k^2 units of time
 - (d) $n - k$ units of time
4. If a half adder takes time t_1 and a full adder takes time t_2 , what is the total computational time required for an n -bit Ripple Carry Adder?
 - (a) $nt_1 + t_2$
 - (b) nt_2
 - (c) $(n - 1)t_1 + t_2$
 - (d) $t_1 + (n - 1)t_2$

2.5 Full Adders and Carry-Lookahead Adders (MCQ)

1. In a full adder circuit, the propagation delays of the 2-input logic gates are given below. The time taken to produce the sum and carry outputs respectively are:

$$t_{\text{XOR}} = 30 \text{ ns}, \quad t_{\text{AND}} = t_{\text{OR}} = 10 \text{ ns}$$

- (a) 30 ns, 40 ns
- (b) 40 ns, 30 ns
- (c) 60 ns, 50 ns
- (d) 60 ns, 60 ns

2. A 4-bit carry-lookahead adder adds two 4-bit numbers. The adder is designed without using XOR gates. The propagation delay for all gates is given as 2.4 ns. What will be the overall delay of the adder if we assume that inputs are made available in both complemented and uncomplemented form, and the carry network has been implemented using AND, OR gates only? (Write your answer only as a floating point number rounded to 1 decimal place. Do not write any units after the answer.)
3. The number of full and half adders required to add two 16-bit numbers is:
 - (a) 8 half adders, 8 full adders
 - (b) 1 half adder, 15 full adders
 - (c) 16 half adders, 0 full adders
 - (d) 4 half adders, 12 full adders
4. In an 8-bit ripple carry adder using identical full adders, each full adder takes 34 ns for computing sum. If the time taken for 8-bit addition is 90 ns, calculate the time taken (in ns) by each full adder to find the carry. (Write your answer only as a single integer. Do not write any units after the answer.)
5. Let $S(x, y, z)$ and $C(x, y, z)$ represent the sum and carry functions of a full adder circuit. Which of the following options best represents $S(x, y, z)$ and $C(x, y, z)$ respectively?
 - (a) $x \oplus y \oplus z, \quad y(x \oplus z) + xy$
 - (b) $x \oplus y \oplus z, \quad y(x + y + z)$
 - (c) $x \odot y \odot z, \quad z(x + y) + xy$
 - (d) None of these
6. A full adder circuit takes 20 ns to generate the carry-out bit and 40 ns for the sum bit. When four 1-bit full adders are cascaded, the maximum rate of additions per second will be $K \times 10^6$. What is the value of K ? (Write your answer only as a single integer.)
7. In a Carry-Lookahead Adder, what is the Propagate function which dictates if an incoming carry will be rippled through to the output?
 - (a) $p_i = A_i \oplus B_i$
 - (b) $p_i = A_i \cdot B_i$
 - (c) $p_i = A_i + B_i$
 - (d) $p_i = \overline{A_i \oplus B_i}$

3 Sequential Logic

3.1 Flip-Flops (MCQ)

1. Why is a JK flip-flop considered more versatile than a D flip-flop? (FF with NAND)
 - (a) It uses fewer gates
 - (b) It can implement SR, D, and T behaviors

- (c) It is asynchronous
 - (d) It needs no clock
2. How many NAND gates are present in a Master–Slave D flip-flop?
- (a) 4
 - (b) 8
 - (c) 12
 - (d) 10
3. The fundamental advantage of a D flip-flop over an SR flip-flop is:
- (a) Faster operation
 - (b) No race condition
 - (c) Elimination of invalid state
 - (d) Lower power consumption
4. Why is a clock added to a basic NAND SR latch?
- (a) To eliminate propagation delay
 - (b) To prevent invalid condition
 - (c) To control when input affects output
 - (d) To increase fan-out
5. In a NAND-based clocked SR latch, when the clock is LOW, the latch:
- (a) Toggles
 - (b) Becomes transparent
 - (c) Holds its previous state
 - (d) Goes to invalid state
6. The slave latch in a master–slave D flip-flop is active when:
- (a) clock = 1
 - (b) clock = 0
 - (c) Both clock = 0 and clock = 1
 - (d) None
7. Which of the following is used for storing one bit?
- (a) Encoder
 - (b) Decoder
 - (c) D flip-flop
 - (d) Mux

8. When $J = 1$ and $K = 0$ in a Master–Slave JK flip-flop (NAND gate), the output:
 - (a) Resets
 - (b) Sets
 - (c) Toggles
 - (d) Holds
9. The major drawback of a clocked NAND latch is:
 - (a) High power
 - (b) Edge-triggered behavior
 - (c) Level-sensitive behavior
 - (d) Cannot store data
10. Why is the Master–Slave D flip-flop (with NAND gate) edge-triggered?
 - (a) Because NAND gates are fast
 - (b) Because data propagates only during clock transition
 - (c) Because clock frequency is high
 - (d) Because feedback is removed

4 Memory Systems

4.1 Memory and Address Bus

1. A processor can support a maximum memory of 4 GB where the memory is word-addressable (2 B). The size of the address bus of the processor is at least _____?
2. A computer with a 32-bit wide data bus uses $4k \times 8$ Static RAM memory chips. The smallest memory this computer can have is _____?

4.2 SRAM (MCQ)

1. Why is a standard read operation slow in a large, unoptimized SRAM array?
 - (a) The inverters inside the cell switch very slowly.
 - (b) The word line decoder introduces massive delays.
 - (c) Long bit lines introduce high resistance and capacitance.
 - (d) The cross-coupled inverters draw too much current.
2. When sensing the voltage difference Δ , what does a positive difference ($\Delta > 0$) signify?

$$\Delta = V(\text{BL}) - V(\overline{\text{BL}})$$

- (a) The cell is empty.
- (b) A logical 0 is stored.

- (c) A logical 1 is stored.
 - (d) The read operation failed due to noise.
3. What is the role of the Word Line in an SRAM cell?
- (a) It transfers the data out of the cell during a read operation.
 - (b) It supplies the constant power (V_{cc}) to the cell.
 - (c) It continuously refreshes the data in the cell.
 - (d) It turns on the access transistors to connect the internal inverters to the bit lines.
4. What is the primary function of the Column Multiplexer in an SRAM memory array?
- (a) To boost the voltage of the word line.
 - (b) To choose a specific subset of columns to read from or write to.
 - (c) To refresh the data periodically across all columns.
 - (d) To decode the row address.
5. In a large array of SRAM cells arranged in rows and columns, what component ensures that only one row is accessed at a specific time?
- (a) Column Multiplexer
 - (b) Sense Amplifier
 - (c) Pre-charge Driver
 - (d) Decoder
6. How is new data written into an SRAM cell?
- (a) By isolating the bit lines and letting the cross-coupled inverters flip randomly.
 - (b) By using strong driver circuits to charge the bit lines to the desired values while the word line is simultaneously enabled.
 - (c) By dropping the word line voltage to 0 V and allowing the cell to reset.
 - (d) By applying a higher V_{cc} directly to the target cell to override its current state.
7. How is the logical state of the SRAM cell inferred quickly during a pre-charged read?
- (a) By waiting for one bit line to fully reach 1 V and the other 0 V.
 - (b) By measuring the current flowing through the word line.
 - (c) By monitoring the voltage difference between the two bit lines using sense amplifiers.
 - (d) By checking the capacitance of the bit line.
8. In a 6-Transistor SRAM cell, how many bit lines are required to read from or write to a single cell?
- (a) 1
 - (b) 2

- (c) 3
 - (d) 4
9. When utilizing the pre-charging technique, what happens immediately after the word line is enabled?
 - (a) Both bit lines instantly jump to 1 V.
 - (b) Both bit lines instantly drop to 0 V.
 - (c) One bit line voltage starts moving towards 0 V, and the other starts moving towards 1 V.
 - (d) The cell's internal state is wiped out.
 10. What is the fundamental storage component within a standard SRAM cell?
 - (a) A cross-coupled pair of NAND gates
 - (b) A cross-coupled pair of inverters
 - (c) A single capacitor and a transistor
 - (d) A combination of multiplexers and decoders
 11. What specific hardware component is responsible for monitoring the tiny voltage difference on the bit line during a read operation?
 - (a) The Word Line Decoder
 - (b) The Column Multiplexer
 - (c) The Sense Amplifier
 - (d) The Refresh Driver

4.3 DRAM (MCQ)

1. How is a logical '1' physically represented within a DRAM cell?
 - (a) By a continuous current flowing through the word line
 - (b) By a stored electrical charge across the cell's capacitor
 - (c) By connecting the transistor directly to ground
 - (d) By the absence of charge in the capacitor
2. What is the primary function of the Word Line when interacting with a DRAM cell?
 - (a) It permanently stores the data bit.
 - (b) It directly amplifies the output voltage.
 - (c) It serves as a switch, enabling the access transistor when driven to a logical 1.
 - (d) It provides the reference voltage for pre-charging.

3. Before a read operation begins, the bit line in a DRAM array must be pre-charged. Assuming a logical '1' is represented by 1.0 V, what is the standard pre-charge voltage applied to the bit line?
 - (a) 1.0 V
 - (b) 0.0 V
 - (c) 2.0 V
 - (d) 0.5 V
4. Because of charge leakage and destructive reads, what critical maintenance protocol must a DRAM system perform?
 - (a) Periodically reading each and every cell and writing the same data back
 - (b) Discharging the entire array to 0 V every millisecond
 - (c) Swapping the data between rows to balance wear
 - (d) Bypassing the sense amplifiers completely
5. What immediately happens to the stored charge in a DRAM cell's capacitor as a direct result of performing a read operation?
 - (a) The charge doubles due to amplification.
 - (b) The charge remains perfectly intact.
 - (c) Some charge is lost because it is shared with the long bit line.
 - (d) The capacitor becomes permanently polarized.

4.4 Memory Cell Comparison (MCQ)

1. Which of the following memory cell types requires the least amount of physical area?
 - (a) Master-Slave D flip-flop
 - (b) SRAM cell
 - (c) DRAM cell
 - (d) Both SRAM and DRAM require the same area

4.5 Memory Hierarchy and Locality (MCQ)

1. Based on the trade-offs discussed in class, what happens if you try to reduce memory access latency?
 - (a) Area decreases and power decreases
 - (b) Area increases and power increases
 - (c) Area decreases and power increases
 - (d) Area increases and power decreases
2. Which of the following is NOT listed as a factor that memory access latency depends on?

- (a) Size of the memory
 - (b) Technology used (e.g., SRAM, DRAM)
 - (c) CPU clock speed
 - (d) Number of ports
3. How is “Spatial Locality” defined?
- (a) If a resource is accessed, it will likely be accessed again in a short period of time.
 - (b) If a resource is accessed, similar resources will most likely be accessed again in the near future.
 - (c) The distance from the top of the memory stack to where an element is found.
 - (d) The physical distance between the L1 cache and the main memory.
4. When evaluating temporal locality in programs using a stack, what does the “stack distance” represent?
- (a) The total size of the stack in bytes.
 - (b) The difference between the memory addresses of the last two accesses.
 - (c) The distance from the top of the stack to where the memory address was found.
 - (d) The number of programs currently running on the processor.
5. “Address distance” (the difference between a current memory address and the closest address in the recent memory accesses) is used to demonstrate which concept?
- (a) Spatial Locality
 - (b) Temporal Locality
 - (c) Memory overlapping
 - (d) Latency trade-offs
6. In a typical cache hierarchy, what type of memory cells are generally used to build the L1 and L2 caches?
- (a) DRAM cells
 - (b) SRAM cells
 - (c) Master–Slave D flip-flops
 - (d) A mix of DRAM and flip-flops
7. In a hierarchical memory system, what type of memory cell is typically used for main memory?
- (a) SRAM cells
 - (b) Master–Slave D flip-flops
 - (c) DRAM cells
 - (d) Flash memory cells

4.6 Cache Organization (MCQ)

1. In a fully associative cache, a memory block can be placed:
 - (a) Only in one fixed location
 - (b) In any cache entry
 - (c) Only in L1 cache
 - (d) Only in main memory
2. In a direct mapped cache, a memory block maps to:
 - (a) Any location
 - (b) Multiple locations
 - (c) Exactly one location
 - (d) Only L2 cache
3. The index in a direct mapped cache is typically computed using:
 - (a) Tag bits
 - (b) Offset bits
 - (c) Block address modulo number of entries
 - (d) Random selection
4. A set associative cache with k blocks per set is called:
 - (a) k -level cache
 - (b) k -bit cache
 - (c) k -way associative cache
 - (d) k -index cache
5. In a k -way set associative cache, how many locations are checked during lookup?
 - (a) 1
 - (b) k
 - (c) All cache entries
 - (d) None
6. In a fully associative cache, which component is typically used for tag comparison?
 - (a) SRAM array
 - (b) DRAM cells
 - (c) CAM (Content Addressable Memory)
 - (d) Register file

7. In a typical cache hierarchy, what type of memory cells are generally used to build the L1 and L2 caches?
 - (a) DRAM cells
 - (b) SRAM cells
 - (c) Master–Slave D flip-flops
 - (d) A mix of DRAM and flip-flops
8. What happens when a cache miss occurs?
 - (a) Data is discarded
 - (b) CPU halts permanently
 - (c) Data is fetched from lower memory levels and cache is updated
 - (d) Cache is cleared

4.7 Memory Cells (Long-form)

1. (a) State the fundamental physical components used to store a single bit of information in a 6-transistor SRAM cell versus a DRAM cell.
- (b) Explain why a DRAM array requires periodic refreshing while an SRAM array does not. What specific steps occur during a DRAM refresh operation?
- (c) In an array of SRAM cells, reading is optimized using a technique called pre-charging. Briefly explain the physical issue this technique solves and how it achieves faster read times.

5 Instruction Set Architecture and Microprogrammed Control

5.1 Instruction Formats

1. A machine has a 32-bit architecture, with 1-word long instructions. It has 64 registers, each 32 bits long. It needs to support 45 instructions which have an immediate operand in addition to two register operands. Assuming the immediate operand is a signed two's complement integer, what is the range of the immediate operand?
2. A processor has 128 registers and uses a 32-bit instruction format. It has two types of instructions: I-type and R-type. Each I-type instruction contains an opcode, a register name, and an 8-bit immediate value. Each R-type instruction contains an opcode and two register names. If there are 32 distinct I-type opcodes, then what is the maximum number of distinct R-type opcodes?
3. A processor has 16 integer registers (R0, R1, ..., R15) and 64 floating-point registers (F0, F1, ..., F63). It uses a 2-byte instruction format. There are four categories of instructions:
 - **Type-1:** four instructions, each with 3 integer register operands (3R).
 - **Type-2:** eight instructions, each with 2 floating-point register operands (2F).

- **Type-3:** fourteen instructions, each with one integer register and one floating-point register operand (1R + 1F).
- **Type-4:** N instructions, each with one floating-point register operand (1F).

What is the maximum value of N ?

4. Consider a processor with 128 registers and an instruction set of size fourteen. Each instruction has five distinct fields: opcode, two source register identifiers, one destination register identifier, and a 12-bit immediate value. Each instruction must be stored in memory in a byte-aligned fashion. If a program has 100 instructions, what is the amount of memory (in bytes) consumed by the program text?
5. A machine has a 32-bit architecture, with 1-word long instructions. It has 64 registers, each 32 bits long. It needs to support 45 instructions which have an immediate operand in addition to two register operands. Assuming the immediate operand is an unsigned integer, what is the maximum value of the immediate operand?
6. Consider a processor with 64 registers and an instruction set of size twelve. Each instruction has five distinct fields: opcode, two source register identifiers, one destination register identifier, and a 12-bit immediate value. Each instruction must be stored in memory in a byte-aligned fashion. If a program has 100 instructions, what is the amount of memory (in bytes) consumed by the program text?
7. A processor has 16 integer registers (R0, R1, ..., R15) and 64 floating-point registers (F0, F1, ..., F63). It uses a 2-byte instruction format. There are four categories of instructions:
 - **Type-1:** four instructions, each with 3 integer register operands (3R).
 - **Type-2:** eight instructions, each with 2 floating-point register operands (2F).
 - **Type-3:** fourteen instructions, each with one integer register and one floating-point register operand (1R + 1F).
 - **Type-4:** N instructions, each with one floating-point register operand (1F).

What is the maximum value of N ? (Variant: same problem statement.)

5.2 PC-Relative Addressing (Long-form)

1. Consider a hypothetical ISA where each instruction is exactly 4 bytes long. Conditional and unconditional branch instructions in this ISA use PC-relative addressing mode with offset specified in bytes to the target location of the branch instruction. Further, the offset is always with respect to the address of the next instruction in the program sequence. Also, the `add` and `sub` instructions follow similar definitions and syntax as SimpleRISC, while the `cmp` and `beq` instructions are defined as:
 - `cmp R1, R2, R3`: This instruction compares the values stored in registers R2 and R3. The result of this comparison is stored in register R1, which acts as a condition register for subsequent control flow.
 - `beq R1, Offset`: This is a Branch-if-Equal instruction. It examines the comparison result stored in register R1; if the condition for equality is met, program execution jumps

to the target address calculated using PC-relative addressing with the offset specified in the instruction.

Now consider the following instruction sequence:

Instr. No.	Instruction
i	add R2, R3, R4
$i + 1$	sub R5, R6, R7
$i + 2$	cmp R1, R9, R10
$i + 3$	beq R1, Offset

If the target of the branch instruction is i , what is the decimal value of the offset? (2 marks)

5.3 Microprogrammed Control Units

1. Consider a single address field format used for branch control logic. Assume the control word is 64 bits wide. The control portion of the micro-instruction format is divided into two fields: a micro-operation field of 24 bits specifies the micro-operations to be performed, and an address selection field specifies a condition based on the flags that will cause a micro-instruction branch. There are 16 flags. How many bits are in the address field? If the memory is byte-oriented, what is the size of the memory?
2. Consider a CPU with 128 instructions in the instruction set, and each operation requires 8 cycles to finish. If 60 control signals need to be generated by the control unit, and a single address field format is used, what is the minimum size of the control word in a horizontal microprogrammed control unit?
3. Consider a hypothetical control unit which supports 512 bytes of control word memory. If the control word is 20 bits long and the hardware contains 4 flags and supports 16 branch conditions, what is the maximum number of control signals that can be generated at a time using horizontal microprogramming and vertical microprogramming, respectively?
4. Consider a microprogrammed control unit that supports 7 different mutually exclusive groups of control signals as given below:

Control Groups	G1	G2	G3	G4	G5	G6	G7
Control Signals	2	10	4	1	18	23	6

What is the minimum number of control bits required in horizontal and vertical microprogramming?

5. Consider a CPU with 256 instructions in the instruction set, and each operation requires 10 cycles to finish. If 100 control signals need to be generated by the control unit, and a single address field format is used, what is the minimum size of the control word in the horizontal and vertical microprogrammed control unit?
6. Consider a hypothetical control unit which supports 1024 bytes of control word memory. If the control word is 30 bits long and the hardware contains 4 flags and supports 16 branch conditions, what is the maximum number of control signals that can be generated at a time using horizontal microprogramming and vertical microprogramming, respectively?

6 SimpleRisc Processor

6.1 SimpleRisc Pipeline (MCQ)

1. In the SimpleRisc 5-stage pipeline, which stage is responsible for both breaking the instruction into fields and computing the branch target address ($PC + \text{offset}$)?
 - (a) Execute (EX)
 - (b) Instruction Fetch (IF)
 - (c) Memory Access (MA)
 - (d) Operand Fetch (OF)
2. In the Instruction Fetch (IF) stage of the SimpleRisc processor, the PC register is negative-edge triggered. If the current instruction is at address A and a branch is not taken, what is the value of the PC after the next negative clock edge?
 - (a) $A + 1$
 - (b) $A + 2$
 - (c) $A + 4$
 - (d) $A + 8$
 - (e) None of the above
3. A processor has a 5-stage pipeline: IF, OF, EX, MA, RW. If a `call` instruction is executed, in which stage is the return address written to the register file?
 - (a) EX
 - (b) OF
 - (c) MA
 - (d) RW
 - (e) IF
4. In the ALU multiplexer logic, if the instruction is `add rd, rs1, rs2`, what is the value of the `isImmediate` control signal?
 - (a) 0
 - (b) 1
 - (c) Depends on the value of `rs2`
 - (d) It is not used for `add`
5. If the PC is currently at `0x00001000` and a branch instruction `b` with an offset of `0x00000005` is executed, what is the `branchTarget`? (Assume the offset is a word offset and the calculation is $PC + (\text{offset} \ll 2)$.)
 - (a) `0x00001005`
 - (b) `0x00001010`

- (c) 0x00001014
 - (d) 0x00001020
6. Inside the ALU, the `isCmp` signal is active. What internal hardware component is primarily used, and what is updated as a result?
- (a) Multiplier; updates `op1`
 - (b) Adder; updates the flags register
 - (c) Shift unit; updates `rd`
 - (d) Logical unit; updates the PC
 - (e) None of the above
7. In a hardwired control unit, the control signals are functions of the opcode and the state. If the processor is in the Operand Fetch stage, and the opcode corresponds to a `st` (Store) instruction, what must the `isSt` signal be?
- (a) High impedance
 - (b) Undefined until the EX stage
 - (c) 1
 - (d) 0
8. If `flags.E = 0` and `flags.GT = 1`, and the instruction is `bgt` (Branch if Greater Than), what is the value of `isBranchTaken`?
- (a) 0
 - (b) 1
 - (c) $A + B$
 - (d) None of the above
9. Which of the following instructions would cause the Memory Access (MA) stage to actively interface with the data memory system?
- (a) `add`
 - (b) `cmp`
 - (c) `st`
 - (d) `mov`
10. The `branchTarget` calculation involves shifting an offset and adding it to the PC. If the branch offset is 27 bits, why is it shifted by 2 bits before addition?
- (a) To convert a word-offset into a byte-address.
 - (b) To sign-extend the value to 32 bits.
 - (c) To account for the 2-bit modifier.
 - (d) To multiply the offset by 2.

6.2 SimpleRisc Instruction Encoding and Control (MCQ)

1. A `bgt` instruction is fetched from memory address `0x08000020`. If the branch condition is met, the next instruction fetched is from `0x0800004C`. What is the exact 27-bit binary offset embedded within the instruction encoding?
 - (a) `0...01011`
 - (b) `0...10110`
 - (c) `0...01100`
 - (d) `0...2A000`
2. An `add` instruction utilizes an 18-bit immediate value of `0x2F1A4`. During the Operand Fetch (OF) stage, this immediate is sign-extended to 32 bits to produce `immx`. What is the 32-bit hexadecimal value of `immx` supplied to the ALU?
 - (a) `0x0002F1A4`
 - (b) `0xFFFFEF1A4`
 - (c) `0x8002F1A4`
 - (d) `0xFFFFF1A40`
3. Assume register `r4` contains the value `0x00004000`. The instruction `ld r3, -8[r4]` is currently executing. During the Memory Access (MA) stage, what exact 32-bit value is placed into the Memory Address Register (MAR)?
 - (a) `0x00003FF8`
 - (b) `0x00004008`
 - (c) `0xFFFFFFFF8`
 - (d) `0x00004000`
4. The processor fetches a 32-bit instruction with the hexadecimal encoding `0x78000000`. Based strictly on decoding the top 5 bits, what will be the boolean values of the `isSt`, `isLd`, and `isWb` control signals in the OF stage?
 - (a) `isSt=0, isLd=1, isWb=1`
 - (b) `isSt=1, isLd=0, isWb=1`
 - (c) `isSt=1, isLd=0, isWb=0`
 - (d) `isSt=0, isLd=0, isWb=0`
5. A `call` instruction is located at address `0x1000A004`. During the RW stage of this instruction, a multiplexer routes a specific value to the write data port of the register file. What is the precise numerical value written, and to which register address?
 - (a) `0x1000A004` to `rd`
 - (b) `0x1000A008` to `ra(15)`
 - (c) `0x1000A008` to `rd`

(d) `0x1000A004` to `ra(15)`

6. Consider the hardwired control logic for the `isUbranch` signal given below. Which of the following opcodes will NOT evaluate to 1 for this equation?

$$\text{isUbranch} = op_5 \cdot \overline{op_4} \cdot (\overline{op_3} \cdot op_2 + op_3 \cdot \overline{op_2} \cdot \overline{op_1})$$

(a) 10010

(b) 10011

(c) 10100

(d) 10000

7. According to the provided control unit boolean logic, the `isCall` signal is defined simply as `isCall = op5`. Given that the opcode for `beq` is 10000, what unintended hardware conflict would this simplified boolean equation cause during the execution of a `beq` instruction?

(a) It would force the ALU to perform an addition instead of a comparison.

(b) It would overwrite the return address register `ra(15)` with `PC + 4` in the RW stage.

(c) It would cause the IF multiplexer to unconditionally take the branch.

(d) It would stall the pipeline in the OF stage.

8. Evaluate the `isWb` boolean expression for a `nop` instruction (opcode 01101), as given below. What does this expression mathematically reduce to for this specific instruction?

$$\text{isWb} = \sim (op_5 + \overline{op_5} \cdot op_3 \cdot op_1 \cdot (op_4 + \overline{op_2})) + op_5 \cdot op_4 \cdot op_3 \cdot op_2 \cdot op_1$$

(a) $1 + 0 = 1$

(b) $0 + 1 = 1$

(c) $\sim (1) + 0 = 0$

(d) $\sim (0) + 0 = 1$

9. The `isRet` signal serves as the selector for two distinct multiplexers across different pipeline stages. Which two values are ultimately brought face-to-face at the final IF stage PC multiplexer when `isRet = 1`?

(a) `branchTarget` and `PC + 4`

(b) The contents of `ra(15)` and `PC + 4`

(c) `immx` and `PC + 4`

(d) `aluResult` and `branchTarget`

10. If `flags.E = 1` and `flags.GT = 0`, and the fetched instruction is `bgt 0x00000008`, what arithmetic calculation ultimately determines the updated Program Counter value in the IF stage?

(a) `PC + 0x00000008`

(b) `PC + 4`

(c) `PC + 0x00000020`

- (d) `ra(15)`
11. To optimize power, the ALU utilizes transmission gates at the inputs of its functional units. If the processor decodes a `cmp r1, r2` instruction, what are the logic states of the control signal S for the transmission gates feeding the Adder and the Logical Unit, respectively?
 - (a) Adder $S = 1$, Logical Unit $S = 1$
 - (b) Adder $S = 0$, Logical Unit $S = 1$
 - (c) Adder $S = 1$, Logical Unit $S = 0$
 - (d) Adder $S = 0$, Logical Unit $S = 0$
 12. An instruction requires calculating an extended immediate value (`immx`) while simultaneously decoding the opcode to generate the `isWb` signal. In which specific pipeline stage boundary do both of these distinct hardware operations physically reside?
 - (a) Instruction Fetch (IF)
 - (b) Operand Fetch (OF)
 - (c) Execute (EX)
 - (d) Between EX and Memory Access (MA)
 13. During the OF stage, the register file receives inputs to its Read Port 2. If the `isSt` (Store) signal is active (1), which bits of the 32-bit instruction format directly dictate the address fed into Read Port 2?
 - (a) Bits 15:18
 - (b) Bits 19:22
 - (c) Bits 23:26
 - (d) Bits 28:32
 14. The `ret` instruction uses the I bit (bit 27) in its encoding. Architecturally, why is the specific value (0 or 1) of the I bit completely irrelevant to the correct execution of the `ret` instruction in the EX stage?
 - (a) The `isRet` multiplexer overrides the ALU output, bypassing the `immx` vs. `op2` selection entirely.
 - (b) The I bit is physically disconnected from the ALU during branch instructions.
 - (c) The ALU forces a logic 0 on all inputs when `isRet = 1`.
 - (d) `ret` does not use the EX stage.
 15. Trace the hardware path of `branchTarget`. Before it reaches the EX stage multiplexer, it is calculated in the OF stage using an Adder. What are the two specific 32-bit inputs to this dedicated OF stage Adder?
 - (a) `op1` and `immx`
 - (b) PC and `immx`
 - (c) PC and Offset (shifted 2 bits and sign-extended)

- (d) $PC + 4$ and Offset

7 Pipelining

7.1 Pipelining Fundamentals (MCQ)

1. What is the primary advantage of pipelining in a processor?
 - (a) It reduces the execution time of a single isolated instruction.
 - (b) It keeps all parts of the data path busy all the time.
 - (c) It eliminates the need for an Execute (EX) unit.
 - (d) It allows multiple instructions to write to the same register simultaneously.
2. In a perfectly balanced 5-stage pipeline, if an instruction takes T seconds to complete without pipelining, how often does a new instruction complete its execution with pipelining?
 - (a) Every T seconds
 - (b) Every $5T$ seconds
 - (c) Every $T/5$ seconds
 - (d) Every T^5 seconds
3. In the 5-stage pipeline design, what component is inserted between consecutive stages and is triggered by a negative clock edge?
 - (a) Multiplexers
 - (b) Latches (Pipeline Registers)
 - (c) Forwarding Units
 - (d) Hardware Interlocks
4. Which of the following data hazards is the ONLY one possible in a strictly in-order pipeline?
 - (a) Write After Write (WAW)
 - (b) Write After Read (WAR)
 - (c) Read After Read (RAR)
 - (d) Read After Write (RAW)
5. What specifically causes a Structural Hazard in a pipelined processor?
 - (a) Two instructions have a conflict on the same set of resources in a given cycle.
 - (b) Instructions from the wrong path of a branch are fetched.
 - (c) An instruction requires data that has not yet been computed.
 - (d) A hardware interrupt forces the pipeline to flush.
6. To solve control hazards in software, compilers can utilize a technique involving “delay slots.” What does this entail?

- (a) Disabling the clock for a few cycles until the branch resolves.
 - (b) Moving valid instructions that must execute anyway into the spaces immediately following a branch instruction.
 - (c) Flushing the entire register file when a branch is detected.
 - (d) Inserting a “Data-Lock” command into the code stream.
7. When implementing a hardware “Data-Lock” interlock to resolve a RAW hazard, which specific pipeline stages must be stalled?
- (a) EX and MA
 - (b) MA and RW
 - (c) IF and OF
 - (d) OF and EX
8. What exactly constitutes a “pipeline bubble” inserted by hardware to handle stalls?
- (a) A temporary hardware buffer.
 - (b) A `nop` (no-operation) instruction packet.
 - (c) A dedicated signal that speeds up the system clock.
 - (d) A mechanism to clear data memory.
9. What is the primary benefit of adding data “forwarding” paths to the pipeline?
- (a) It completely eliminates the need for an Instruction Fetch unit.
 - (b) It passes a needed value from a later stage directly to an earlier stage, reducing or eliminating stalls.
 - (c) It ensures that instructions are always executed out-of-order.
 - (d) It acts as an interlock that stops the clock entirely.
10. According to the rules for creating forwarding paths, which of the following pipeline stages is NEVER a part of any forwarding path?
- (a) Operand Fetch (OF)
 - (b) Memory Access (MA)
 - (c) Execute (EX)
 - (d) Instruction Fetch (IF)

7.2 Pipelined Processor Datapath (MCQ)

1. A processor has a 5-stage pipeline where each stage performs the same amount of work. If a non-pipelined version of the processor completes one instruction every T seconds, what is the exact execution time required to completely process 100 independent instructions in the pipelined processor?
- (a) $100T/5$ seconds

- (b) $100T$ seconds
 - (c) $T + 99(T/5)$ seconds
 - (d) $T/5 + 99T$ seconds
2. To ensure proper isolation and prevent interference between multiple instructions executing simultaneously, an “instruction packet” travels between stages. Based on the system design, which of the following is NOT explicitly defined as a component of this packet traveling across the pipeline latches?
 - (a) Program counter
 - (b) Hardware memory sequence limits
 - (c) Control signals
 - (d) All intermediate results
 3. How is the propagation of these instruction packets between the 5 stages strictly controlled in this datapath?
 - (a) By 5 latches triggered by the positive edge of the clock.
 - (b) By 4 latches triggered by the positive edge of the clock.
 - (c) By 5 latches triggered by the negative edge of the clock.
 - (d) By 4 latches triggered by the negative edge of the clock.
 4. The Operand Fetch (OF) stage is responsible for decoding instructions and preparing the necessary data for execution. Which specific bundle of signals is latched into the OF-EX register to be utilized by the Execute stage?
 - (a) The program counter (**pc**), the instruction itself, the computed **aluResult**, and the control signal bundle.
 - (b) The program counter (**pc**), the computed **branchTarget**, the prepared ALU operands (A and B), the store operand (**op2**), the instruction itself, and the control signal bundle.
 - (c) The **isBranchTaken** flag, the calculated **branchPC**, ALU condition flags, and the prepared ALU operands (A and B).
 - (d) Memory control flags (**isLd**, **isSt**), the memory address register (**mar**), the memory data register (**mdr**), and the control signal bundle.
 5. A pipelined processor heavily relies on control signals traveling alongside data. In which stage is the primary Control Unit located that decodes the raw instruction to generate the control signal bundle?
 - (a) Instruction Fetch (IF) Stage
 - (b) Operand Fetch (OF) Stage
 - (c) Execute (EX) Stage
 - (d) Register Write (RW) Stage

6. In the Execute (EX) stage, the processor must determine if a conditional branch should be taken. What specific outputs from the Arithmetic Logic Unit (ALU) are utilized by the Branch unit to make this logical determination?
 - (a) The computed `branchTarget`
 - (b) The decoded `aluSignals`
 - (c) The generated flags
 - (d) The `isUBranch` control signal
7. When a Store instruction (`isSt` is active) reaches the Memory Access (MA) stage, it must provide an address and data to the Memory unit. Which values originating from the EX-MA pipeline register are mapped to the memory address register (`mar`) and memory data register (`mdr`)?
 - (a) `mar` receives the program counter (`pc`); `mdr` receives the computed `aluResult`.
 - (b) `mar` receives the computed `aluResult`; `mdr` receives the designated store operand (`op2`).
 - (c) `mar` receives the first register operand (`op1`); `mdr` receives the designated store operand (`op2`).
 - (d) `mar` receives the computed `branchTarget`; `mdr` receives the loaded memory result (`ldResult`).
8. During a subroutine call (when `isCall` is active) in the Register Write (RW) stage, which specific value is selected by the multiplexer to be written into the return address register?
 - (a) The `aluResult` passed down from the EX stage.
 - (b) The `ldResult` fetched from the data memory in the MA stage.
 - (c) The instruction's `pc` plus 4, calculated by a dedicated adder located within the RW stage.
 - (d) The `branchTarget` generated dynamically in the OF stage.

7.3 Pipeline Hazards — Detailed Scenarios (MCQ)

1. Consider a 5-stage in-order RISC pipeline consisting of Instruction Fetch (IF), Operand Fetch (OF), Execute (EX), Memory Access (MA), and Register Write (RW) stages. The pipeline does NOT implement data forwarding (bypassing). An instruction writes its result to the register file during the RW stage. A dependent instruction cannot enter the OF stage until the clock cycle after the producer instruction completes its RW stage. For the following code sequence, how many stall cycles (or NOP instructions) must the compiler insert between the two instructions to perfectly resolve the Read-After-Write (RAW) hazard?

```
I1: add r1, r2, r3
I2: sub r4, r1, r5
```

- (a) 1
- (b) 2
- (c) 3

- (d) 4
2. A program containing a loop executes a conditional branch instruction (`beq r1, r2, target`). The processor uses a standard 5-stage pipeline (IF, OF, EX, MA, RW). Branch conditions and target addresses are evaluated and resolved entirely within the Execute (EX) stage. If the branch instruction is fetched in clock cycle 1 and evaluates to “taken”, in which exact clock cycle will the correct target instruction be fetched, assuming the processor does not use branch prediction and simply fetches sequentially until the branch is resolved?
- (a) Cycle 2
(b) Cycle 3
(c) Cycle 4
(d) Cycle 5
3. A compiler is optimizing the instruction sequence below for a 5-stage pipeline (IF, OF, EX, MA, RW) without data forwarding. To prevent data hazards, dependent instructions must be sufficiently separated from their producers. If the compiler is allowed to reorder independent instructions without changing program semantics to minimize wasted cycles, what is the absolute minimum number of NOP instructions it must still insert before I4?

```
I1: add r1, r2, r3
I2: add r4, r5, r6
I3: add r7, r8, r9
I4: sub r10, r1, r4
```

- (a) 3
(b) 2
(c) 1
(d) 0
4. Consider a processor executing the following code snippet:

```
I1: add r1, r2, r3
I2: sub r1, r4, r5
```

If the processor utilizes a strictly in-order pipeline, it will only ever experience Read-After-Write (RAW) data hazards. However, if the processor architecture is upgraded to allow out-of-order execution, what specific type of data hazard could arise between I1 and I2, and what is its fundamental definition?

- (a) A Write-After-Write (WAW) hazard, occurring if I2 writes to `r1` before I1 writes to it.
(b) A Write-After-Read (WAR) hazard, occurring if I2 writes to `r1` before I1 reads it.
(c) A Read-After-Write (RAW) hazard, occurring if I2 reads `r1` before I1 writes to it.
(d) A structural hazard, occurring because both instructions attempt to access the ALU simultaneously.

5. In a standard 5-stage pipeline (IF, OF, EX, MA, RW), an arithmetic instruction `add r1, r2, r3` is fetched in clock cycle 1. During which clock cycle is the correctly computed mathematical result of this addition actively generated inside the Arithmetic Logic Unit (ALU)?
- (a) Cycle 5
 - (b) Cycle 4
 - (c) Cycle 3
 - (d) Cycle 2
6. To mitigate the performance penalty of control hazards, a processor architecture implements a “delayed branch” strategy with two delay slots. Which of the following statements rigorously defines the execution semantics of the instructions placed in these delay slots by the compiler?
- (a) They are evaluated conditionally and are committed to memory/registers only if the branch is taken.
 - (b) They are automatically flushed from the pipeline if the branch target is reached.
 - (c) They must exclusively be NOP instructions to ensure safe pipeline state transitions.
 - (d) They are assumed to be valid program instructions and are executed unconditionally, regardless of whether the branch is taken.
7. As an instruction flows through a 5-stage pipeline (IF, OF, EX, MA, RW), it carries an “instruction packet” containing state information from stage to stage via pipeline latches. Which of the following components is NOT logically present in the data transferred from the Execute (EX) stage into the Memory Access (MA) stage via the EX-MA latch?
- (a) The program counter (PC) of the instruction.
 - (b) The mathematical result computed by the ALU (`aluResult`).
 - (c) The propagated control signals required for subsequent stages.
 - (d) The data fetched from memory (`ldResult`).
8. In a 5-stage RISC architecture, the Control Unit is situated in the Operand Fetch (OF) stage. It decodes the instruction and generates all necessary control signals for execution, memory access, and write-back. For a standard arithmetic instruction, where do these control signals physically travel immediately upon leaving the OF stage?
- (a) Into the OF-EX pipeline latch to travel synchronously with the instruction into the Execute stage.
 - (b) Directly to the Register File to trigger a data write.
 - (c) To the Instruction Fetch (IF) stage to update the branch target buffer.
 - (d) Directly to the Memory Unit to pre-fetch potential data operands.
9. Assume a program executes continuously on a completely un-pipelined processor, taking exactly T seconds to complete one full instruction cycle. If this processor is redesigned into a perfectly balanced 5-stage pipeline where each stage performs an equal fraction of the work, and the pipeline is kept completely full with no hazards, what is the theoretical time interval between the completion of consecutive instructions?

- (a) T seconds
 - (b) $5T$ seconds
 - (c) $T/25$ seconds
 - (d) $T/5$ seconds
10. A structural hazard occurs when two instructions attempt to utilize the same hardware resource in the exact same clock cycle. In a standard 5-stage pipeline (IF, OF, EX, MA, RW) with isolated instruction and data memories, which of the following combinations of simultaneous operations is structurally safe and will NOT cause a hazard?
- (a) Two consecutive instructions attempting to access the ALU in the EX stage.
 - (b) An instruction fetching an operand from the Register File in the OF stage while an older instruction writes a result to the Register File in the RW stage.
 - (c) An instruction fetching from instruction memory in the IF stage while an older instruction loads data from data memory in the MA stage.
 - (d) Both B and C are structurally safe in a standard 5-stage pipeline with dual-ported register files and separate memories.

7.4 Pipeline Mathematical Model (MCQ)

1. According to the mathematical model of an ideal pipeline, how does the optimal number of pipeline stages (k) relate to the latch delay (l)?
 - (a) It is directly proportional to l
 - (b) It is inversely proportional to the square root of l
 - (c) It is directly proportional to l^2
 - (d) It is independent of l
2. As the amount of algorithmic work ($t_{\max}$) increases, what happens to the optimal number of pipeline stages for ideal performance?
 - (a) It decreases
 - (b) It increases
 - (c) It remains constant
 - (d) It drops to 1
3. For an ideal pipeline with n instructions and k stages, what is the formula for the Cycles Per Instruction (CPI)?
 - (a) $(n + k - 1)/n$
 - (b) $(n + k)/k$
 - (c) $n/(n + k - 1)$
 - (d) $(n - 1 + k)/k$

4. In an ideal pipeline model, as the number of instructions (n) tends to infinity, what happens to the ideal number of pipeline stages?
 - (a) It tends to 1
 - (b) It tends to a constant value
 - (c) It tends to infinity
 - (d) It tends to $t_{\max}$
5. In reality, pipelines have stalls. How is the actual CPI of a non-ideal pipeline calculated?
 - (a) $\text{CPI}_{\text{ideal}} - (\text{stall_rate} \times \text{stall_penalty})$
 - (b) $\text{CPI}_{\text{ideal}} \times \text{stall_rate}$
 - (c) $\text{CPI}_{\text{ideal}} / \text{stall_penalty}$
 - (d) $\text{CPI}_{\text{ideal}} + (\text{stall_rate} \times \text{stall_penalty})$
6. In the mathematical model presented for a non-ideal pipeline, the stall penalty is assumed to be proportional to which of the following?
 - (a) The number of instructions (n)
 - (b) The number of pipeline stages (k)
 - (c) The latch delay (l)
 - (d) The frequency (f)
7. According to the non-ideal pipeline model, how should the optimal number of pipeline stages be adjusted for a program with a high number of dependences (a high stall rate, r)?
 - (a) Increase the number of stages
 - (b) Decrease the number of stages
 - (c) Keep the number of stages constant
 - (d) Make the number of stages equal to the number of instructions
8. Compared to a pipeline that just has interlocks, how does adding data forwarding affect the optimal number of pipeline stages?
 - (a) It requires a smaller number of pipeline stages.
 - (b) It requires a larger number of pipeline stages.
 - (c) It does not change the optimal number of pipeline stages.
 - (d) It eliminates the need for pipeline stages altogether.

7.5 Pipeline Execution Problems (Numerical)

1. Consider a non-pipelined processor with a clock rate of 2.5 GHz and an average cycles per instruction (CPI) of 4. The same processor is upgraded to a pipelined architecture with 5 stages. However, due to internal pipeline delays, the clock rate is reduced to 2 GHz. Assume that there are no pipeline stalls. (3 + 3 + 2 marks)

- (i) Calculate the total execution time for 150 instructions in the non-pipelined processor.
 - (ii) Calculate the total execution time for 150 instructions in the pipelined processor.
 - (iii) Determine the speedup achieved by the pipelined processor over the non-pipelined processor.
2. Consider a pipelined processor with the following stages:
- IF: Instruction Fetch
 - ID: Instruction Decode and Operand Fetch
 - EX: Execute
 - WB: Write Back

The IF, ID, and WB stages take one clock cycle each. The number of clock cycles required for the EX stage depends on the instruction type:

- ADD and SUB instructions require 2 clock cycles in the EX stage.
- MUL instruction requires 4 clock cycles in the EX stage.

Assume that operand forwarding is used in the pipeline. Determine the number of clock cycles required to complete the execution of the following instruction sequence:

```
ADD R2, R1, R0    // R2 <- R0 + R1
MUL R4, R3, R2    // R4 <- R3 * R2
SUB R6, R5, R4    // R6 <- R5 - R4
```

(3 marks)

3. Consider a 4-stage pipelined processor with stages S_1, S_2, S_3 , and S_4 . The number of clock cycles required by four instructions I_1, I_2, I_3 , and I_4 in each stage is given in the table below:

	S_1	S_2	S_3	S_4
I_1	2	1	1	1
I_2	1	3	2	2
I_3	2	1	1	3
I_4	1	2	2	2

Determine the total number of clock cycles required to complete the execution of the instruction sequence I_1, I_2, I_3, I_4 . (3 marks)

4. The baseline execution time of a program on a 2 GHz single-core machine is 100 nanoseconds (ns). The portion of the code corresponding to 90% of the execution time can be fully parallelized. The overhead for using each additional core is 10 ns when running on a multicore system. Assume that all cores execute their share of the parallelized portion equally. Determine the number of cores that minimizes the total execution time of the program. (3 marks)
5. Consider a 3-stage pipelined processor with stage delays of 10 ns, 20 ns, and 14 ns for the first, second, and third stages, respectively. Assume that there are no additional delays and the processor does not suffer from any pipeline hazards. Also assume that one instruction is

fetched every cycle. Determine the total execution time (in nanoseconds) required to execute 100 instructions on this processor. (3 marks)

8 Assembly Programming

1. Write an assembly code for calculating factorial using a loop.
2. Write an assembly code for calculating factorial using recursion.
3. Write assembly-level code for generating the Fibonacci sequence.
4. A number is known as a *cubic Armstrong number* if the sum of the cubes of its individual decimal digits is equal to the number itself. For example, 153 is a cubic Armstrong number ($153 = 1^3 + 5^3 + 3^3$).

You are given a number in register r_0 , and it is known to be between 1 and 1,000,000. Write a piece of assembly code in SimpleRisc to determine if this number is a cubic Armstrong number. Save 1 in register r_1 if it is a cubic Armstrong number; otherwise, save 0. (5 marks)

9 Theory of Computation

9.1 Turing Machines

1. **Problem TM.1: Incrementing a Number.** Design a deterministic Turing Machine

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{acc}}, q_{\text{rej}})$$

that increments a non-negative integer by 1.

Input format.

- The input number is written in binary.
- The tape contains $\$b_{k-1} b_{k-2} \cdots b_0\$$ where $b_i \in \{0, 1\}$, and b_0 is the least significant bit (LSB).
- The tape head initially points to the LSB b_0 .

Output. After halting, the tape should contain the binary representation of the input number plus one.

2. **Problem TM.2: Checking for a Palindrome.** Design a deterministic Turing Machine

$$M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{acc}}, q_{\text{rej}})$$

that decides whether a string over $\{a, b\}$ is a palindrome.

Input format.

- Tape contains $\$w\$$ where $w \in \{a, b\}^*$.
- Head initially points to the rightmost symbol.

Output. Enter q_{acc} iff w is a palindrome.